

Reddit Pain Scout

Project Specification & Build Brief

One-line summary: An automated tool that scans Reddit for people complaining about specific problems and emails a daily digest of qualified leads to founders and solopreneurs looking for their next customers or their next product idea.

1. Main Motive & Why We're Building This

Solo founders and indie hackers constantly need two things: validated problems to build products around, and real leads to talk to once a product exists. The most reliable signal for both already exists publicly on Reddit — people openly venting about problems they're facing, in detail, without the filter they'd apply on LinkedIn or X. The motive behind this project is to turn that existing signal into an automated, recurring source of qualified leads and validated ideas, instead of relying on manual, time-consuming scrolling.

2. The Problem (Before This Tool)

- Founders manually scroll dozens of subreddits daily hoping to stumble on a relevant complaint — a reported 15–20 hours per week for some solopreneurs.
- There is no reliable way to filter Reddit for genuine pain points versus casual mentions, memes, or unrelated noise.
- By the time a relevant post is found manually, it may already be old, buried, or the poster has moved on.
- Existing tools either monitor broadly (e.g. generic social listening tools) and are too expensive/enterprise-focused, or don't exist at all for this specific use case.

3. What It Solves

- Removes the need to manually scroll Reddit — the tool watches chosen subreddits and keywords continuously.
- Filters out noise so only genuine complaints/pain points reach the user, not every keyword mention.
- Delivers results proactively (via email) instead of requiring the user to check a dashboard.
- Surfaces warm leads — people already expressing a need — rather than cold, unqualified prospects.

4. How It Solves It

The tool combines scheduled data collection, lightweight natural language filtering, and automated email delivery into a single pipeline. Instead of a human doing the searching, reading, and judging, the pipeline does it automatically every day and only surfaces what passed the relevance filter. The user's only manual step is reading one short email and deciding who to reply to.

5. Workflow — How It Works, Step by Step

Step 1 — Configuration

The user (or the tool owner, at MVP stage) defines which subreddits to watch (e.g. r/Entrepreneur, r/SaaS, r/Solopreneur, r/smallbusiness) and which keywords/phrases represent the pain points they care about.

Step 2 — Scheduled fetch

A scheduled job (triggered externally via cron-job.org, calling a Next.js API route) runs daily. It authenticates with Reddit's OAuth2 API and pulls new posts/comments from each configured subreddit.

Step 3 — Deduplication

The system checks each post's unique Reddit ID against previously seen IDs stored in the database, so the same post is never processed twice across runs.

Step 4 — Pain filtering

Each new post is passed through a filtering step (keyword scoring and/or a lightweight AI classification call) that determines whether it represents a genuine complaint or pain point, discarding irrelevant or noisy mentions.

Step 5 — Ranking

Posts that pass the filter are ranked by relevance and recency, and capped to a reasonable daily digest length.

Step 6 — Email delivery

The ranked results are compiled into a digest email and sent via Resend to the user's inbox.

Step 7 — Action

The user reads the digest and replies directly to relevant Reddit threads with genuine, non-spammy responses connecting to the problem the poster described.

6. Tech Stack

Layer	Technology	Purpose
Framework	Next.js (App Router)	Frontend + API routes, single deployable app
Scheduling	cron-job.org (external)	Triggers the daily digest API route on a schedule
Database / ORM	MongoDB + Prisma	Stores users, projects, tracked keywords, seen post IDs, digest history
Auth	NextAuth / JWT + Google OAuth	User accounts and session management
Reddit data access	Reddit API (OAuth2) via snoowrap	Authenticated fetching of subreddit posts/comments
Filtering	Keyword scoring / lightweight AI classification	Distinguishes genuine complaints from noise
Email delivery	Resend	Sends the daily digest email
Billing	Stripe / Freemius	Subscription plans and payment handling

Hosting	Vercel	Deployment of the Next.js app and API routes
---------	--------	--

7. High-Level Architecture

cron-job.org → Next.js API route (/api/cron/digest) → Reddit API fetch → Deduplication check (MongoDB via Prisma) → Pain filter → Ranking → Resend email delivery → User inbox

The API route is protected by a secret bearer token so only cron-job.org (holding the correct Authorization header) can trigger it, preventing abuse of the Reddit API quota or unwanted repeated email sends.

8. Project Folder Structure

```

app/
  api/
    cron/
      digest/
        route.ts - endpoint triggered by cron-job.org
lib/
  reddit.ts - Reddit OAuth + fetch logic (snoowrap)
  filter.ts - pain-point filtering logic
  digest.ts - ranking + email composition
  email.ts - Resend integration
prisma/
  schema.prisma - database schema (MongoDB)

```

9. Final Product — Full Feature Set

This is the complete, end-state version of the product — a proper multi-user SaaS, not a single-user script. Every feature below is part of the finished vision.

- User accounts — sign up / log in with email or Google OAuth, each user has their own isolated workspace.
- Subreddit & keyword configuration — users manage their own tracked subreddits and keywords from a settings page, no hardcoding.
- Multiple tracked "projects" per user — e.g. one set of subreddits/keywords per product or niche, each with its own digest.
- Live lead feed — a dashboard showing every matched post in real time, not just what arrived in the last email.
- Daily digest email — automated, branded email summarizing the day's best matches, sent via Resend.
- Relevance scoring & manual feedback — users can mark a result as "useful" or "not relevant" to improve future filtering.
- Reply tracking — mark leads as "contacted" / "replied" / "ignored" so nothing is followed up on twice.

- Subscription billing — free tier (limited subreddits/keywords) and paid tiers (more projects, more frequent scans) via Stripe or Freemius, consistent with the AutoPilot billing pattern.
- Usage analytics — simple charts showing matches found per day/week and digest open rates.
- Account & notification settings — digest frequency (daily/twice-daily), email on/off, timezone for delivery time.

10. User Interface — Screens & Layout

The final product is a clean, dashboard-style SaaS, visually consistent with modern indie SaaS tools (Tremor/Shadcn-style components, generous whitespace, a single accent color). Below is every screen in the finished app.

Landing page

A single marketing page explaining the product in one sentence, followed by a short 3-step "how it works" visual, a pricing section (Free / Pro), and a prominent "Get started" button. Social-proof style testimonials or example digest screenshots sit above the fold to build trust quickly.

Sign up / Login

A minimal auth screen with email/password and "Continue with Google" options, matching the OAuth pattern already used in AutoPilot.

Onboarding (first-run)

A short 3-step wizard: (1) name your project, (2) pick subreddits from a searchable list, (3) enter keywords/phrases. Ends with "Your first digest arrives tomorrow at 8 AM" confirmation.

Main dashboard

The core screen after login. A left sidebar lists the user's projects. The main panel shows a live feed of matched Reddit posts as cards — each card shows the subreddit, post title/snippet, relevance score, and quick actions (Mark useful / Mark contacted / Open on Reddit). A top filter bar allows sorting by date, subreddit, or status.

Project settings

A form-based page per project to edit subreddits, keywords, digest frequency, and delivery time. Includes a "pause project" toggle.

Analytics page

Simple charts (using Recharts, consistent with prior dashboard work) showing matches per day, filter accuracy feedback, and digest email open rates over time.

Billing / plan page

Shows current plan, usage against plan limits (number of projects/subreddits), and an upgrade button leading to Stripe/Freemius checkout, mirroring the tiered billing pattern from AutoPilot.

Account settings

Basic profile info, notification preferences, and the option to delete account/data.

Daily digest email (outside the app)

A clean, branded HTML email listing the day's top matches as a simple list — subreddit, one-line excerpt, and a direct link to the Reddit thread. A footer link goes straight back to the dashboard.

This document describes the complete, final vision of the product — every feature and screen intended to exist in the finished app. It is meant as a full build brief to hand to a coding assistant (e.g. Claude Code) to scaffold and implement the project described above.